

Internship Project Report

On

“IoT-Based AQI Monitoring System”

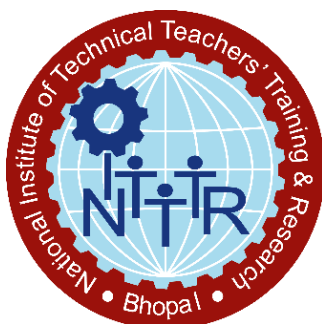
Submitted in partial fulfilment of requirement for the award of a certificate of internship programme

In

National Institute of Technical Teacher's Training and Research, Bhopal

Deemed to be University (under distinct category)

Submitted to



Deemed to be University under
Distinct Category

Submitted by

MAANYA MATHUR

Under the Supervision of

Mr. ANKIT KUMAR

Centre of Excellence

June 2026

**National Institute of Technical Teachers' Training & Research, Bhopal
Deemed to be University (Under Distinct Category)**

Ministry of Education, Govt. of India

Shamla Hills, Bhopal - 462 002 (M.P.), India

**National Institute of Technical Teachers' Training and Research,
Bhopal**

Deemed to be University (under distinct category)

INTERNSHIP PROJECT REPORT

CERTIFICATE

This is to certify that the work embodies in this dissertation entitled
“IOT-BASED AQI MONITORING SYSTEM” being submitted by
“MAANYA MATHUR” for partial fulfilment of the requirement for the
award of **“Certificate”** during the short-term internship programme is a
record of a Bonafide piece of work, which was carried out by him under my
supervision and guidance in the of **Centre of Excellence**

National Institute of Technical Teachers' Training and Research, Bhopal

Supervisor
Prof. Sanjeet Kumar

National Institute of Technical Teachers' Training and Research, Bhopal
Deemed to be University (under distinct category)

CANDIDATE'S DECLARATION

I, **MAANYA MATHUR** hereby declare that the report entitled “**IOT-BASED AQI MONITORING SYSTEM**” which is being submitted to Centre of Excellence in **National Institute of Technical Teachers' Training and Research, Bhopal**, is our authentic work carried out during internship.

I declare that my work has not been submitted in part or in full to any other university or institution for the award of any certificate.

MAANYA MATHUR
(JECRC UNIVERSITY)

National Institute of Technical Teachers' Training and Research, Bhopal
Deemed to be University (under distinct category)

ACKNOWLEDGEMENT

I am sincerely thankful to the National Institute of Technical Teacher' Training and Research, Bhopal for giving me the opportunity to pursue this internship in the Centre of Excellence. I would like to express my heartfelt gratitude to Mr. Ankit Kumar for his valuable guidance and continuous support during the course of this internship.

The learning and exposure I received during the internship will be instrumental in shaping my future endeavours.

MAANYA MATHUR
(JECRC UNIVERSITY)

National Institute of Technical Teachers' Training and Research, Bhopal
Deemed to be University (under distinct category)

Table of Content

S.NO	Content	Page No.
1	<u>INSTALLATION AND HARDWARE SETUP</u> 1.1 COMPONENTS REQUIRED 1.2 SYSTEM ARCHITECTURE	6 6 6
2	<u>COMPONENT DESCRIPTION & PIN CONNECTIONS</u> 2.1 ESP32 AND ARDUINO NANO MICROCONTROLLER 2.2 PMS 5003 2.3 BME 680 2.4 MQ135 2.5 MAX4466 2.6 P10 PANEL 2.7 SMPS 5V 40 A 2.8 PERF BOARD	6 7-8 9 10 11 12 14 15 16
3	<u>CIRCUIT DIAGRAM & FINAL CODE</u>	17-38
4	<u>DATA FLOW & LOCAL SERVER</u> 4.1 DATA FLOW 4.2 LOCAL SERVER 4.2.1 HIVE MQ 4.2.2 NODE RED 4.2.3 COMMUNICATION PROTOCOL	38 38 39 39 39 40

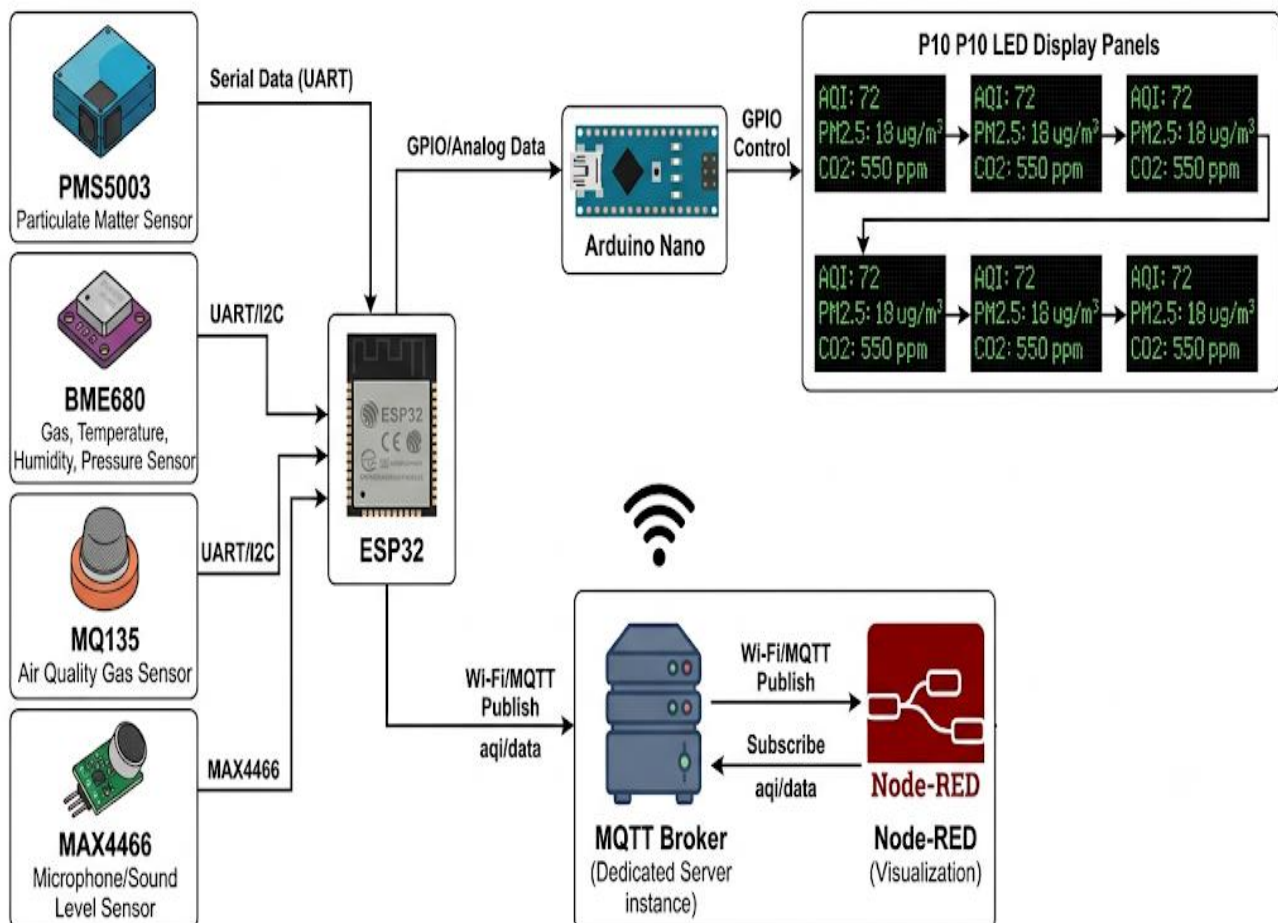
1. INSTALLATION AND HARDWARE SETUP

1.1 COMPONENTS REQUIRED

- ESP32 & ARDUINO NANO MICROCONTROLLER
- PSM5003
- BME680
- MQ135
- MAX4466
- P10 PANEL
- SMPS 5V 40A
- PERF BOARD

1.2 SYSTEM ARCHITECTURE

SYSTEM ARCHITECTURE: IOT BASED AQI MONITORING SYSTEM



2. COMPONENT DESCRIPTION & PIN CONNECTIONS

2.1 ESP32 AND ARDUINO NANO MICROCONTROLLER

- **ESP32:** The ESP32 is a low-cost, low-power **System-on-Chip (SoC)** microcontroller developed by Espressif Systems. It is designed for Internet of Things (IoT) applications and wearable electronics, standing out due to its integrated **Wi-Fi** and **dual-mode Bluetooth** capabilities.



WORKING: The ESP32 operates on a **dual-core architecture** combined with a Real-Time Operating System (RTOS):

- **Dual-Core Processing:** It contains two 32-bit CPUs. Core 0 typically manages wireless communication (Wi-Fi/Bluetooth stacks), while Core 1 executes the main project application code simultaneously.
- **Task Prioritization:** Instead of running code strictly line-by-line like traditional boards, it uses FreeRTOS to handle multiple tasks asynchronously based on priority.

HOW IT WORKS:

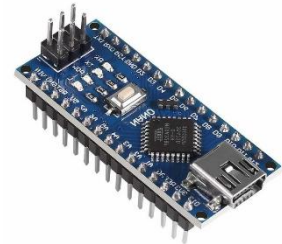
- Powering Up: The board runs on 3.3V logic. An onboard regulator steps down 5V USB power to a stable 3.3V.
- Code Execution: The bootloader loads the programmed code from the external flash memory into the high-speed internal SRAM for the processor to execute
- Data I/O: The ESP32 reads inputs from sensors via its General-Purpose Input/Output (GPIO) pins using an internal 12-bit Analog-to-Digital Converter (ADC).
- Wireless Gateway: The collected data is formatted into internet protocols (like HTTP or MQTT) and sent via Wi-Fi or Bluetooth to a cloud server or mobile application.

PIN CONNECTIONS:

ESP32 PIN	CONNECTED TO
GPIO 22	SCL(BME680)
G21	SDA(BME680)
G16	TX(PMS5003)
G17	RX(PMS5003)
G4	D10(NANO)
V5	POWER SUPPLY
GND	GND(POWER SUPPLY)

G32	AO(MQ135)
G34	OUT(MAX4466)

- **ARDUINO NANO:** The Arduino Nano is a small, complete, and breadboard-friendly microcontroller board based on the ATmega328P processor. Developed by Arduino.cc, it packs the same computational power and functionality as the larger Arduino Uno but in a highly compact form factor, making it ideal for space-constrained, low-power electronic projects and robotics.



WORKING: The Arduino Nano operates on a single-core, synchronous execution principle:

- **Sequential Processing:** It contains a single 8-bit RISC processor that executes instructions one line at a time in a rigid, sequential sequence at a clock speed of 16 MHz.
- **Harvard Architecture:** It utilizes separate memory pathways for data (SRAM) and instructions (Flash Memory), allowing the processor to fetch and execute code efficiently without lag

HOW IT WORKS:

- Powering Up:** The board runs on **5V logic**. It can be powered via a Mini-B USB cable or through an unregulated 6-20V external power source connected to the VIN pin, which an onboard regulator steps down to 5V.
- Code Execution:** Upon boot, the ATmega328P executes the `setup ()` function once to configure the pins, and then repeats the code written inside the `loop ()` function indefinitely.
- Data I/O:** The Nano reads voltages from physical sensors via its digital and analog input pins (using an internal 10-bit Analog-to-Digital Converter) to evaluate real-world conditions.
- Control Output:** Based on the code logic, the microcontroller sends signals out to components like LEDs, motors, or relays using its GPIO pins or Pulse Width Modulation (PWM) to perform physical actions.

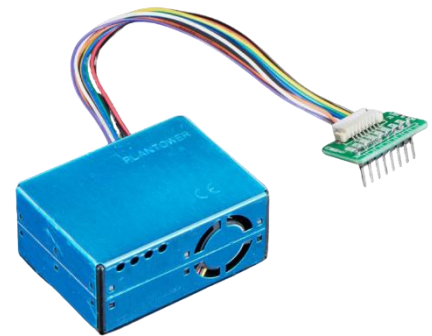
PIN CONNECTIONS:

ARDUINO NANO PIN	CONNECTED TO
---------------------	--------------

D10	G4(ESP32)
GND	GND(P10)
D13	CLK(P10)
D6	A(P10)
D7	B(P10)
D8	LAT(P10)
D9	OE(P10)
D11	PR(P10)

2.2 PMS5003

The **PMS5003** is a digital, universal particle concentration sensor used to measure airborne particulate matter (**PM1.0, PM2.5, and PM10**). It uses a built-in fan to draw in air and provides real-time mass concentration data, making it ideal for air quality monitoring systems.



WORKING: The sensor operates on the principle of laser scattering:

- **Mie Scattering:** A laser diode shines light into a specialized chamber. When airborne particles pass through the beam, they scatter the light.
- **Optical Detection:** A photodetector measures the intensity and frequency of the scattered light, which is directly proportional to the size and number of the particles.

HOW IT WORKS:

- Air Intake:** A built-in miniature fan continuously pulls ambient air into the sensor's inlet and vents it through the outlet, ensuring a constant, uniform airflow sample.
- Optical Scanning:** As the air stream passes through the internal chamber, the laser continuously hits the moving particles, generating rapid light pulses.
- Signal Processing:** The photodetector converts the light pulses into weak electrical signals. An internal amplifier boosts these signals, and a digital signal processor (DSP) counts and sizes the particles.

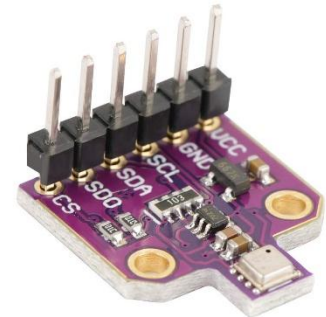
- iv. Data Transmission: The processed data is packed into a 32-byte frame containing mass concentration values for PM1.0, PM2.5, and PM10. This data is transmitted out sequentially at a 9600 baud rate using **UART serial communication** (TX/RX pins), which can be easily read by microcontrollers like the ESP32 or Arduino Nano.

PIN CONNECTIONS:

PMS5003 PIN	CONNECTED TO
VCC	5V(POWER SUPPLY)
GND	GND(POWER SUPPLY)
RX	G17(ESP32)
TX	G16(ESP32)

2.3 BME680

The **BME680** is an advanced, low-power **4-in-1 digital environmental sensor** developed by Bosch Sensortec. Unlike standard sensors that only measure one or two variables, the BME680 integrates high-linearity and high-accuracy sensors for **gas, pressure, humidity, and temperature** into a single, microscopic package. It is heavily utilized in smart home devices, weather stations, and IoT air-quality monitoring projects due to its ability to calculate an Indoor Air Quality (IAQ) index.



WORKING: The BME680 operates by combining multiple distinct micro-electro-mechanical systems (MEMS) technologies on a single silicon chip:

- **Piezoresistive & Capacitive Sensing:** Temperature, atmospheric pressure, and relative humidity are measured using traditional MEMS structures. Changes in the physical environment alter the electrical resistance or capacitance of these internal microscopic structures.
- **Metal-Oxide (MOx) Chemiresistor:** The gas sensor works on a chemical change principle. It contains an internal hotplate that heats up a metal-oxide layer (usually Tin Dioxide, SnO_2). When Volatile Organic Compounds (VOCs) or polluting gases (like carbon monoxide or alcohol) contact this heated layer, a chemical reaction occurs that alters the material's electrical resistance.

HOW IT WORKS:

- i. Dynamic Heating: When a measurement begins, the sensor quickly pulses its internal hotplate to temperatures between 200°C and 400°C (depending on the settings) to prepare the gas-sensing layer.

- ii. **Physical Data Collection:** Simultaneously, the ambient air alters the internal capacitive and piezoresistive tracks to capture the current temperature, humidity, and barometric pressure.
- iii. **Chemical Resistance Testing:** As air passes over the heated metal-oxide layer, polluting gas molecules alter the oxide's surface chemistry, changing its resistance. The sensor measures this specific change in electrical resistance to determine gas concentration.
- iv. **Digital Conversion and Output:** An internal 16-bit Analog-to-Digital Converter (ADC) translates all these raw physical and chemical variations into precise digital values. The BME680 then transmits this compiled data to a microcontroller (like an ESP32 or Arduino Nano) using standard **I2C** or **SPI** communication protocols

PIN CONNECTIONS:

BME680 PIN	CONNECTED TO
VCC	3.3V(POWER SUPPLY)
GND	GND(POWER SUPPLY)
SDA	G21(ESP32)
SCL	GPIO22(ESP32)

2.4 MQ135

The MQ135 is a low-cost, highly popular analog and digital semiconductor sensor used to monitor atmospheric conditions. It is specifically designed to detect a **broad spectrum of harmful gases and air pollutants**, including Ammonia (NH_3), Nitrogen Oxides (NO_x), Alcohol, Benzene, Carbon Dioxide (CO_2), and smoke. Because it responds to multiple contaminants simultaneously, it is heavily used as a general-purpose **Indoor Air Quality (IAQ)** monitor in smart ventilation systems, homes, and offices.

WORKING : The MQ135 operates on the principle of Surface Adsorption and Resistance Change using a Metal-Oxide Semiconductor (MOS):

- **Sensing Material:** At its core, the sensor uses Tin Dioxide (SnO_2), a semiconductor material that naturally exhibits high electrical resistance (low conductivity) when exposed to clean, fresh air.
- **Chemical Adsorption:** When the surrounding air contains polluting or reducing gases, those target gas molecules physically bond (adsorb) onto the surface of the heated Tin Dioxide layer.



- **Resistance Drop:** This surface chemical reaction alters the charge density within the semiconductor material, dramatically increasing its electrical conductivity (reducing its resistance). The drop in resistance is directly proportional to the concentration of pollutants in the air.

HOW IT WORKS:

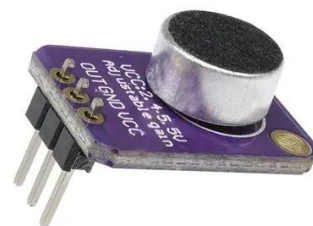
- Preheating:** The sensor features an internal nichrome heating coil wrapped around a ceramic tube. When supplied with 5V power, this coil heats the SnO_2 layer up to an optimal operating temperature (between 200°C and 400°C) required for chemical reactions to occur efficiently.
- Gas Interaction:** As ambient air drifts through the explosion-proof stainless steel protective mesh, any airborne toxic gases react with the heated SnO_2 layer, lowering its internal electrical resistance.
- Voltage Divider Circuit:** The module pairs this variable sensor resistance with a fixed **load resistor (R_L)** to form a simple voltage divider. As the sensor resistance decreases due to heavy pollution, the voltage across the load resistor increases
- Dual Data Output: * Analog Output (AO):** Outputs a continuously varying **0V to 5V signal** that rises alongside the gas concentration, allowing microcontrollers like an ESP32 or Arduino Nano to calculate precise Parts-Per-Million (PPM) trend mapping

PIN CONNECTIONS:

MQ135 PIN	CONNECTED TO
VCC	5V (POWER SUPPLY)
GND	GND (POWER SUPPLY)
AO	G32(ESP32)

2.5 MAX4466

The **MAX4466** is a high-performance, low-power electret microphone amplifier module. It combines a miniature electret condenser microphone with a specialized operational amplifier chip (the Maxim Integrated MAX4466). This module is engineered specifically for audio processing tasks, such as voice changers, audio recording/sampling, and sound-reactive IoT projects. It is highly valued for its excellent power supply noise rejection, which prevents electrical hums from distorting the audio signal.



WORKING: The MAX4466 operates on the principles of Acoustic-to-Electrical Conversion and Op-Amp Signal Amplification:

- **Electret Condenser Sensing:** The onboard microphone contains a lightweight conductive diaphragm placed close to a fixed backplate, forming a capacitor. Sound waves hit the diaphragm, causing it to vibrate. This movement changes the distance between the plates, altering the capacitance and generating a tiny, fluctuating electrical voltage.
- **Operational Amplification:** The raw voltage from the microphone capsule is far too weak (often just a few millivolts) for a standard microcontroller to read accurately. The MAX4466 operational amplifier intercepts this weak signal, utilizing an adjustable feedback loop to scale the voltage up to a usable line-level amplitude.

HOW IT WORKS:

- i. **Sound Capture:** Sound waves from the environment physically vibrate the internal electret membrane, producing a minute alternating current (AC) audio signal.
- ii. **Gain Tuning:** The module features a small, onboard trimmer potentiometer (trimpot) on the back. By turning this screw, you change the resistance in the feedback loop, allowing you to manually adjust the amplification gain from 25x to 125x depending on your project's environmental volume.
- iii. **DC Biasing:** The amplifier adds a direct current (DC) bias offset to the audio wave. Since microcontrollers cannot naturally read negative voltages, the MAX4466 centers the output audio wave perfectly at half of the supply voltage ($VCC/2$), ensuring the entire waveform stays positive.
- iv. **Analog Output Processing:** The final, amplified analog voltage is output through the OUT pin. A microcontroller (like an Arduino Nano or ESP32) can sample this pin using its internal Analog-to-Digital Converter (ADC) to track sound frequencies, measure volume levels, or analyze audio patterns.

PIN CONNECTIONS:

MAX4466 PIN	CONNECTED TO
VCC	3.3 V (POWER SUPPLY)
GND	GND (POWER SUPPLY)
OUT	G32 (ESP32)

2.6 P10 PANEL

The **P10 LED Panel** is a modular, high-brightness digital display unit widely used to create large-scale LED video walls, moving text billboards, and electronic notice boards. The "P10" designation indicates a **pixel pitch of 10mm**, meaning the distance from the center of one LED pixel to the center of the next is exactly 10mm. It is highly favored in outdoor and indoor engineering projects due to its modular design, which allows multiple panels to be daisy-chained together to form massive custom displays.



WORKING: The P10 panel operates on the principles of Multiplexed LED Matrix Driving and Pulse Width Modulation (PWM):

- **Dot-Matrix Structure:** The panel is structured as a grid of Light Emitting Diodes (typically 32 columns by 16 rows, totaling 512 LEDs per panel).
- **Dynamic Multiplexing (Scanning):** To save power and reduce the number of control pins needed, the panel does not light up all LEDs simultaneously. Instead, it uses a scanning method (commonly a 1/4 or 1/8 scan rate). It lights up rows sequentially at an incredibly high speed (hundreds of times per second). Because of human persistence of vision, the human eye perceives a single, continuous flicker-free image.
- **Shift Register Data Transfer:** The panel relies on serial-in, parallel-out shift registers (like the 74HC595). The host controller sends image data serially, and the shift registers convert it into parallel signals to drive the columns.

HOW IT WORKS:

- i. Data Reception: The P10 panel connects to a microcontroller (such as an Arduino Nano or ESP32) using a standardized HUB75 or HUB12 serial interface data bus
- ii. Row and Column Control: The microcontroller sends a clock signal and high-speed binary data streaming

down the rows and columns. Row driver ICs select which row is currently active, while column drivers dictate which LEDs in that row should turn on.

- iii. **Brightness Regulation:** By utilizing PWM signals from the microcontroller, the time duration that an individual LED stays turned on during its scanning microsecond window is altered. This varying duty cycle changes the perceived brightness of individual pixels.
- iv. **Power Architecture:** P10 panels require significant current to power hundreds of LEDs and operate on an external **5V DC power supply**. While the high-current power line goes directly to the panel, the low-current logic lines interface with the microcontroller to synchronize text, graphics, or animations.

PIN CONNECTIONS:

P10 PIN	CONNECTED TO
A	D6 (NANO)
B	D7 (NANO)
CLK	D13(NANO)
LAT	D8(NANO)
PR	D11(NANO)
OE	D9(NANO)
GND	GND(NANO)
V-	V-(SMPS)
V+	V+(SMPS)

2.7 SMPS 5V 40A

A **Switched-Mode Power Supply (SMPS)** is an electronic power unit that utilizes a switching regulator to efficiently convert electrical power from one form to another. A **5V 40A SMPS** specifically takes high-voltage alternating current from mains electricity (typically 110V/220V AC) and converts it into a stable, low-voltage direct current (**5V DC**). Delivering up to **40 Amps** of current, it generates a substantial **200 Watts** of total power, making it the primary power source for heavy 5V hardware architectures like multi-panel **P10 LED displays** and dense sensor grids.

WORKING: The SMPS operates on the principle of High-Frequency Pulse Width Modulation (PWM) and Duty Cycle Control, which makes it vastly more efficient than older linear power supplies:

- **High-Speed Switching:** Instead of dropping excess voltage across a power-wasting resistor, it passes input power through a fast semiconductor

switch (like a Power MOSFET). This switch flips fully "ON" (saturation) and fully "OFF" (cut-off) at ultrasonic frequencies—typically between 20 kHz and 1 MHz.

- **Energy Storage & Transfer:** When the switch is ON, electrical energy flows into energy-storing components like inductors and high-capacity capacitors. When the switch cuts off, the stored energy is gradually released to the load.
- **PWM Regulation:** An internal feedback circuit constantly samples the output voltage. If the output voltage fluctuates or the load draws more current, a PWM controller shifts the ratio of the ON-to-OFF time (the duty cycle) in microseconds to keep the output locked exactly at 5V.

HOW IT WORKS:

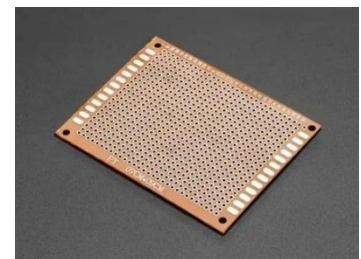
- i. Input Rectification and Filtering: The raw high-voltage AC mains power enters the unit and passes through an EMI filter to remove electrical noise. It then travels through a bridge rectifier and a filtering capacitor to produce an unregulated, high-voltage DC bus line.
- ii. High-Frequency Inversion: The power MOSFET splits this steady high-voltage DC line into a rapid series of square-wave pulses at high frequencies
- iii. Isolation and Step-Down: These high-frequency pulses are fed into the primary winding of a compact high-frequency transformer. Operating at such high speeds allows the transformer to be miniature compared to heavy 50Hz conventional transformers. The transformer scales the voltage down to 5V while safely isolating the high-voltage AC mains from the low-voltage electronics.
- iv. Output Rectification and Feedback Loop: The stepped-down AC pulses are rectified once more by fast-recovery diodes and smoothed out by output capacitors to generate a clean, ripple-free 5V DC output. Simultaneously, an internal optocoupler feeds isolation signals back to the primary controller to trigger over-voltage, over-current, and short-circuit protections if necessary

2.8 PERF BOARD

A **Perfboard** (or DOT PCB) is a rigid material used for prototyping electronic circuits permanently. It features a pre-drilled grid of holes spaced at standard **2.54 mm (0.1 inch)** intervals, designed to perfectly fit standard microcontroller headers, IC pins, and electronic components.

WORKING: The board operates on the principle of isolated mechanical support:

- **Independent Pads:** Each hole is ringed by an isolated copper pad. The surrounding fiberglass or resin substrate is non-conductive, meaning no electrical connections exist until they are manually created.



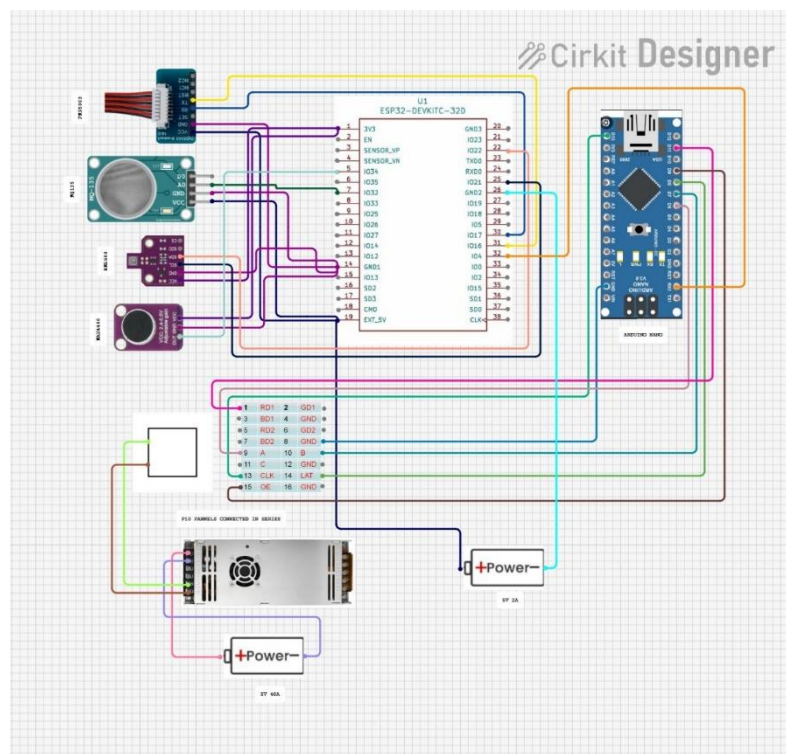
- **Point-to-Point Routing:** Components are held in place by soldering their pins to individual copper rings. Circuit pathways are then manually built on the underside using jumper wires or solder bridges.

HOW IT WORKS:

- Placement:** Components (like the Arduino Nano, ESP32, or sensors) are pushed through the top, non-copper side of the board.
- Soldering:** Solder is melted onto the bottom copper pads, physically locking the component leads in place.
- Wiring:** Insulated hookup wires are soldered between the pins to route electrical signals according to the circuit schematic.
- Testing:** Once point-to-point connections are completed and tested with a multimeter for continuity, the board acts as a durable, custom circuit module.

3. CIRCUIT DIAGRAM AND FINAL CODE

• CIRCUIT DIAGRAM



• FINAL CODE

```
/*
 * ESP32 - 4 Sensor Unified Environmental Monitor
 * + HiveMQ Cloud MQTT publishing
```

```

* + Nano forwarding (Pre-formatted Screen Text Stream)
* + WiFiManager and OTA Updates
*/

#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <WiFiManager.h>
#include <ArduinoOTA.h>
#include <esp_task_wdt.h>
#include "soc/soc.h"
#include "soc/rtc_cntl_reg.h"
#include "time.h"
#include <math.h>

#define MQTT_MAX_PACKET_SIZE 512

#include <PubSubClient.h>
#include <MQUnifiedsensor.h>
#include <HardwareSerial.h>
#include <Wire.h>
#include <Adafruit_BME680.h>

//=====
// HiveMQ Cloud Details
//=====
const char* mqtt_server =
"4b2031eb1d3349bb8e3010d2b7208cae.s1.eu.hivemq.cloud";
const int mqtt_port = 8883;
const char* mqtt_user = "aqinitttr";
const char* mqtt_password = "Nitttr@2026";
const char* topic = "esp32/environment";

WiFiClientSecure espClient;
PubSubClient client(espClient);

HardwareSerial nanoSerial(1);

// —— MQ-135 Config


---




---


#define MQ135_BOARD      "ESP32"
#define MQ135_PIN        32
#define MQ135_TYPE       "MQ-135"
#define MQ135_VOLTAGE    3.3
#define MQ135_ADC_BITS   12
#define RATIO_MQ135_CLEAN_AIR 3.6

```

```
MQUnifiedsensor MQ135(MQ135_BOARD, MQ135_VOLTAGE,  
MQ135_ADC_BITS, MQ135_PIN, MQ135_TYPE);
```

```
bool  mq135_calibrated  = false;  
bool  mq135_cal_started = false;  
int    mq135_cal_count  = 0;  
float  mq135_cal_sum    = 0.0;  
uint32_t mq135_cal_timer = 0;  
uint32_t mq135_read_timer = 0;  
#define MQ135_CAL_INTERVAL 500  
#define MQ135_READ_INTERVAL 3000
```

```
float global_co2 = 0, global_nh3 = 0, global_voc = 0, global_nox = 0, global_ch4  
= 0;
```

```
// ——— PMS5003 Config
```

```
HardwareSerial pmsSerial(2);
```

```
struct PMS5003Data {  
  uint16_t pm1_0;  
  uint16_t pm2_5;  
  uint16_t pm10;  
};  
uint32_t pms_read_timer = 0;  
#define PMS_READ_INTERVAL 2000  
PMS5003Data pmsData = {0, 0, 0};  
int global_aqi = 0;
```

```
// ——— BME680 Config
```

```
Adafruit_BME680 bme;  
uint32_t bme_read_timer = 0;  
#define BME_READ_INTERVAL 2000  
float global_temp = 0, global_hum = 0, global_press = 0;
```

```
// ——— MAX4466 Config
```

```
#define MAX4466_PIN      34  
uint32_t sound_read_timer = 0;  
#define SOUND_READ_INTERVAL 2000  
int global_sound_peak = 0;
```

```
// —— Nano Screen Transmission Config
```

```
// Nano now only displays pre-formatted text; ESP32 owns all  
formatting/switching logic.
```

```
uint32_t screen_timer = 0;
```

```
#define SCREEN_INTERVAL 4000
```

```
int currentScreen = 0;
```

```
const int TOTAL_SCREEN = 22; // 10 original screens + 12 notebook-sequence  
screens
```

```
// —— OTA State
```

```
volatile bool otaInProgress = false;
```

```
// —— NTP / Time Config (IST, UTC+5:30)
```

```
const char* ntpServer = "pool.ntp.org";
```

```
const long gmtOffset_sec = 19800;
```

```
const int daylightOffset_sec = 0;
```

```
const char* daysUpper[7] =
```

```
{"SUN", "MON", "TUE", "WED", "THU", "FRI", "SAT"};
```

```
const char* monthsUpper[12] =
```

```
{"JAN", "FEB", "MAR", "APR", "MAY", "JUN", "JUL", "AUG", "SEP", "OCT", "NOV",  
"DEC"};
```

```
// —— AQI Trend Tracking
```

```
int previousAQIforTrend = -1;
```

```
// —— MQTT Publish Config
```

```
unsigned long lastMsg = 0;
```

```
#define MQTT_PUBLISH_INTERVAL 3000
```

```
unsigned long lastHeapPrint = 0;
```

```
// —— Reliability: Watchdog + Scheduled Restart
```

```
// These are additive safety nets only — they don't change sensor/WiFi/MQTT  
behavior.
```

```
// Watchdog: auto-recovers the board if loop() ever hangs (e.g. a sensor library  
stall).
```

```

// Daily restart: clears any long-term heap fragmentation from weeks of continuous
uptime.
#define WDT_TIMEOUT_SEC    15
#define RESTART_INTERVAL_MS (24UL * 60UL * 60UL * 1000UL) // 24
hours

const char* mqttStateToString(int state) {
    switch (state) {
        case -4: return "MQTT_CONNECTION_TIMEOUT";
        case -3: return "MQTT_CONNECTION_LOST";
        case -2: return "MQTT_CONNECT_FAILED";
        case -1: return "MQTT_DISCONNECTED";
        case 0: return "MQTT_CONNECTED";
        case 1: return "MQTT_CONNECT_BAD_PROTOCOL";
        case 2: return "MQTT_CONNECT_BAD_CLIENT_ID";
        case 3: return "MQTT_CONNECT_UNAVAILABLE";
        case 4: return "MQTT_CONNECT_BAD_CREDENTIALS";
        case 5: return "MQTT_CONNECT_UNAUTHORIZED";
        default: return "UNKNOWN";
    }
}

void flushPMS() {
    while (pmsSerial.available()) pmsSerial.read();
}

bool readPMS(PMS5003Data &d) {
    if (pmsSerial.available() < 32) return false;
    if (pmsSerial.read() != 0x42) { flushPMS(); return false; }
    if (pmsSerial.read() != 0x4D) { flushPMS(); return false; }

    uint8_t buf[30];
    pmsSerial.readBytes(buf, 30);

    uint16_t calcSum = 0x42 + 0x4D;
    for (int i = 0; i < 28; i++) calcSum += buf[i];
    uint16_t recvSum = (buf[28] << 8) | buf[29];
    if (calcSum != recvSum) {
        flushPMS();
        return false;
    }

    d.pm1_0 = (buf[8] << 8) | buf[9];
    d.pm2_5 = (buf[10] << 8) | buf[11];
    d.pm10 = (buf[12] << 8) | buf[13];

```

```

    return true;
}

int calcAQI(float pm2_5) {
    if (pm2_5 <= 12.0) return (int)map((long)pm2_5, 0, 12, 0, 50);
    if (pm2_5 <= 35.4) return (int)map((long)pm2_5, 12, 35, 51, 100);
    if (pm2_5 <= 55.4) return (int)map((long)pm2_5, 35, 55, 101, 150);
    if (pm2_5 <= 150.4) return (int)map((long)pm2_5, 55, 150, 151, 200);
    return 201;
}

void reconnectMQTT() {
    if (!client.connected()) {
        Serial.print("Connecting to HiveMQ... ");
        String clientId = "ESP32-" + WiFi.macAddress();
        clientId.replace(":", "");

        if (client.connect(clientId.c_str(), mqtt_user, mqtt_password)) {
            Serial.println("Connected!");
        } else {
            Serial.print("Failed. State = ");
            Serial.println(mqttStateToString(client.state()));
        }
    }
}

void handleMQ135Calibration() {
    if (mq135_calibrated || !mq135_cal_started) return;
    if (millis() - mq135_cal_timer >= MQ135_CAL_INTERVAL) {
        mq135_cal_timer = millis();
        MQ135.update();
        mq135_cal_sum += MQ135.calibrate(RATIO_MQ135_CLEAN_AIR);
        mq135_cal_count++;
        if (mq135_cal_count >= 10) {
            float r0 = mq135_cal_sum / 10.0;
            if (isinf(r0) || r0 == 0) { mq135_cal_started = false; return; }
            MQ135.setR0(r0);
            mq135_calibrated = true;
            Serial.println("[MQ135] Calibration complete.");
        }
    }
}

void handleMQ135Read() {
    if (!mq135_calibrated) return;
    if (millis() - mq135_read_timer < MQ135_READ_INTERVAL) return;

```

```

mq135_read_timer = millis();

MQ135.update();

float val;
MQ135.setA(110.47); MQ135.setB(-2.862); val = MQ135.readSensor();
global_co2 = (isnan(val) || isinf(val)) ? 0.0 : val;
MQ135.setA(102.2); MQ135.setB(-2.473); val = MQ135.readSensor();
global_nh3 = (isnan(val) || isinf(val)) ? 0.0 : val;
MQ135.setA(34.66); MQ135.setB(-3.369); val = MQ135.readSensor();
global_voc = (isnan(val) || isinf(val)) ? 0.0 : val;
MQ135.setA(44.95); MQ135.setB(-3.445); val = MQ135.readSensor();
global_nox = (isnan(val) || isinf(val)) ? 0.0 : val;
MQ135.setA(605.18); MQ135.setB(-3.937); val = MQ135.readSensor();
global_ch4 = (isnan(val) || isinf(val)) ? 0.0 : val;
}

void handlePMS() {
  if (millis() - pms_read_timer < PMS_READ_INTERVAL) return;
  pms_read_timer = millis();
  if (readPMS(pmsData)) {
    global_aqi = calcAQI(pmsData.pm2_5);
  }
}

void handleBME680() {
  if (millis() - bme_read_timer < BME_READ_INTERVAL) return;
  bme_read_timer = millis();
  if (bme.performReading()) {
    global_temp = bme.temperature;
    global_hum = bme.humidity;
    global_press = bme.pressure / 100.0;
  }
}

void handleMAX4466() {
  if (millis() - sound_read_timer < SOUND_READ_INTERVAL) return;
  sound_read_timer = millis();

  unsigned int signalMax = 0;
  unsigned int signalMin = 4095;
  uint32_t start = millis();

  while (millis() - start < 50) {
    int v = analogRead(MAX4466_PIN);
    if (v < 4095) {

```

```

        if (v > signalMax) signalMax = v;
        if (v < signalMin) signalMin = v;
    }
}

unsigned int peakToPeak = signalMax - signalMin;
int db = map(peakToPeak, 10, 1500, 25, 95);
if (db < 20) db = 20;
if (db > 95) db = 95;

global_sound_peak = db;
}

char* trimLeadingSpaces(char* str) {
    while (*str == ' ') {
        str++;
    }
    return str;
}

// ——— Time/Date string builder for the TIME/DATE loop screen


---


void getTimeDateStrings(char* timeStr, size_t tsz, char* dateStr, size_t dsz) {
    struct tm timeinfo;
    if (!getLocalTime(&timeinfo, 200)) {
        strncpy(timeStr, "TIME N/A", tsz - 1); timeStr[tsz - 1] = '\0';
        strncpy(dateStr, "--- ---", dsz - 1); dateStr[dsz - 1] = '\0';
        return;
    }
    int hour12 = timeinfo.tm_hour % 12;
    if (hour12 == 0) hour12 = 12;
    const char* ampm = (timeinfo.tm_hour < 12) ? "AM" : "PM";
    snprintf(timeStr, tsz, "%d:%02d %s %s", hour12, timeinfo.tm_min, ampm,
daysUpper[timeinfo.tm_wday]);
    snprintf(dateStr, dsz, "%02d %s %d", timeinfo.tm_mday,
monthsUpper[timeinfo.tm_mon], timeinfo.tm_year + 1900);
}

// ——— CPCB-style AQI category (India National AQI bands)


---


const char* getAQICategoryCPCB(int aqi) {
    if (aqi <= 50) return "GOOD";
    if (aqi <= 100) return "SATISFAC";
    if (aqi <= 200) return "MODERATE";
    if (aqi <= 300) return "POOR";
    if (aqi <= 400) return "VERY POOR";
}

```



```

    return "SEVERE";
}

// ——— AQI trend vs the previous time this screen was shown


---


const char* computeTrend(int currentAQI) {
    const char* trend;
    if (previousAQIforTrend < 0) {
        trend = "STABLE";
    } else {
        int diff = currentAQI - previousAQIforTrend;
        if (diff > 5) trend = "RISING";
        else if (diff < -5) trend = "FALLING";
        else trend = "STABLE";
    }
    previousAQIforTrend = currentAQI;
    return trend;
}

// ——— Feels-like temperature (Australian BOM apparent-temp formula, no wind
term) ———
float computeFeelsLike(float tempC, float humPct) {
    float e = (humPct / 100.0) * 6.105 * exp((17.27 * tempC) / (237.7 + tempC));
    return tempC + 0.33 * e - 4.00;
}

// ——— Simple comfort banding from temp + humidity


---


const char* computeComfort(float tempC, float humPct) {
    if (tempC >= 20 && tempC <= 28 && humPct >= 30 && humPct <= 60) return
"GOOD";
    return "POOR";
}

// ——— Boot-time screen sender: prints one packet, then holds it on-screen ———
void sendBootScreen(const char* upper, const char* lower, unsigned long holdMs)
{
    nanoSerial.print("<");
    nanoSerial.print(upper);
    nanoSerial.print("|");
    nanoSerial.print(lower);
    nanoSerial.println(">");
    delay(holdMs);
}

```

```

// —— WiFiManager callback: fires only if the device has to open the config
portal ——
void configModeCallback(WiFiManager *myWiFiManager) {
    sendBootScreen("WIFI PORTAL", "AQI_NITTTR", 200);
    Serial.println("[WiFiManager] Config portal opened");
}
//=====
// Screen Formatting + Nano Transmission
// (Replaces old raw-CSV sendDataToNano(); Nano now just displays these two
lines)
//=====
void sendScreenToNano() {
    if (millis() - screen_timer < SCREEN_INTERVAL) return;
    screen_timer = millis();

    char cUpper[32] = "DATA WAIT";
    char cLower[32] = "SYSTEM LIVE";
    char fBuf1[12], fBuf2[12];

    switch (currentScreen) {
        case 0:
            strcpy(cUpper, "WELCOME TO");
            strcpy(cLower, "NITTTR BHOPAL");
            break;

        case 1:
            dtostrf(isnan(global_temp) ? 0.0 : global_temp, 4, 1, fBuf1);
            snprintf(cUpper, sizeof(cUpper), "TEMP:%sC", trimLeadingSpaces(fBuf1));
            snprintf(cLower, sizeof(cLower), "HUMID:%d%%", (int)(isnan(global_hum) ?
0.0 : global_hum));
            break;

        case 2:
            snprintf(cUpper, sizeof(cUpper), "SOUND:%ddB", global_sound_peak);
            if (global_sound_peak <= 30) {
                strcpy(cLower, "NOISE:QUIET");
            } else if (global_sound_peak <= 65) {
                strcpy(cLower, "NOISE:MODERATE");
            } else if (global_sound_peak <= 85) {
                strcpy(cLower, "NOISE:NOISY");
            } else {
                strcpy(cLower, "NOISE:LOUD!");
            }
            break;

        case 3:

```

```

    snprintf(cUpper, sizeof(cUpper), "AQI:%d", global_aqi);
    snprintf(cLower, sizeof(cLower), "PM1.0:%dug", pmsData.pm1_0);
    break;

case 4:
    snprintf(cUpper, sizeof(cUpper), "PM2.5:%dug", pmsData.pm2_5);
    snprintf(cLower, sizeof(cLower), "PM10:%dug", pmsData.pm10);
    break;

case 5:
    dtostrf(global_co2, 4, 0, fBuf1);
    dtostrf(global_nh3, 3, 1, fBuf2);
    snprintf(cUpper, sizeof(cUpper), "CO2:%sppm", trimLeadingSpaces(fBuf1));
    snprintf(cLower, sizeof(cLower), "NH3:%sppm", trimLeadingSpaces(fBuf2));
    break;

case 6:
    dtostrf(global_voc, 3, 1, fBuf1);
    dtostrf(global_nox, 3, 1, fBuf2);
    snprintf(cUpper, sizeof(cUpper), "VOC:%sppm", trimLeadingSpaces(fBuf1));
    snprintf(cLower, sizeof(cLower), "NOx:%sppm", trimLeadingSpaces(fBuf2));
    break;

case 7:
    dtostrf(global_ch4, 3, 1, fBuf1);
    snprintf(cUpper, sizeof(cUpper), "CH4:%sppm", trimLeadingSpaces(fBuf1));
    snprintf(cLower, sizeof(cLower), "BARO:%dhPa", (int)(isnan(global_press) ?
0.0 : global_press));
    break;

case 8:
    strcpy(cUpper, "AIR QUALITY");
    if (global_aqi <= 50) {
        strcpy(cLower, "ENV: HEALTHY");
    } else if (global_aqi <= 100) {
        strcpy(cLower, "ENV:MODERATE");
    } else if (global_aqi <= 150) {
        strcpy(cLower, "STAY INDOOR");
    } else {
        strcpy(cLower, "ENV:HAZARD");
    }
    break;

case 9:
    strcpy(cUpper, "SYS HEALTH");
    if (global_temp == 0.0 && global_press == 0.0) {

```

```

        strcpy(cLower, "ERR: BME680");
    } else if (pmsData.pm1_0 == 0 && pmsData.pm2_5 == 0 && pmsData.pm10
== 0) {
        strcpy(cLower, "ERR: PMS5003");
    } else {
        strcpy(cLower, "ALL SENSORS OK");
    }
    break;

```

// — Notebook-sequence screens (new)

```

case 10: // A: WELCOME TO / NITTTR

```

```

    strcpy(cUpper, "WELCOME TO");
    strcpy(cLower, "NITTTR");
    break;

```

```

case 11: { // B: TIME / DATE

```

```

    char timeStr[20], dateStr[20];
    getTimeDateStrings(timeStr, sizeof(timeStr), dateStr, sizeof(dateStr));
    strncpy(cUpper, timeStr, sizeof(cUpper) - 1); cUpper[sizeof(cUpper) - 1] = '\0';
    strncpy(cLower, dateStr, sizeof(cLower) - 1); cLower[sizeof(cLower) - 1] =
'\0';
    break;
}

```

```

case 12: { // C: AQI category + TREND

```

```

    const char* cat = getAQICategoryCPCB(global_aqi);
    const char* trend = computeTrend(global_aqi);
    snprintf(cUpper, sizeof(cUpper), "AQI:%d %s", global_aqi, cat);
    snprintf(cLower, sizeof(cLower), "TREND:%s", trend);
    break;
}

```

```

case 13: // D: PM2.5 / PM10

```

```

    snprintf(cUpper, sizeof(cUpper), "PM2.5:%dug", pmsData.pm2_5);
    snprintf(cLower, sizeof(cLower), "PM10:%dug", pmsData.pm10);
    break;

```

```

case 14: // E: Advisory (reuses the existing ENV logic from case 8, repositioned
here)

```

```

    strcpy(cUpper, "AIR QUALITY");
    if (global_aqi <= 50) {
        strcpy(cLower, "ENV: HEALTHY");
    } else if (global_aqi <= 100) {
        strcpy(cLower, "ENV: MODERATE");
    } else if (global_aqi <= 150) {

```

```

        strcpy(cLower, "STAY INDOOR");
    } else {
        strcpy(cLower, "ENV:HAZARD");
    }
    break;

case 15: { // F: FEEL / COMFRT
    float feels = computeFeelsLike(isnan(global_temp) ? 0.0 : global_temp,
    isnan(global_hum) ? 0.0 : global_hum);
    dtostrf(feels, 4, 1, fBuf1);
    snprintf(cUpper, sizeof(cUpper), "FEEL:%sC", trimLeadingSpaces(fBuf1));
    snprintf(cLower, sizeof(cLower), "COMFRT:%s",
    computeComfort(global_temp, global_hum));
    break;
}

case 16: // G: TEMP / HUM
    dtostrf(isnan(global_temp) ? 0.0 : global_temp, 4, 1, fBuf1);
    snprintf(cUpper, sizeof(cUpper), "TEMP:%sC", trimLeadingSpaces(fBuf1));
    snprintf(cLower, sizeof(cLower), "HUM:%d%%", (int)(isnan(global_hum) ?
    0.0 : global_hum));
    break;

case 17: // H: PRES / SOUND
    dtostrf(isnan(global_press) ? 0.0 : global_press, 5, 1, fBuf1);
    snprintf(cUpper, sizeof(cUpper), "PRES:%shPa", trimLeadingSpaces(fBuf1));
    snprintf(cLower, sizeof(cLower), "SOUND:%d.0dB", global_sound_peak);
    break;

case 18: // I: NOISE LEVEL (standalone)
    strcpy(cUpper, "NOISE LEVEL:");
    if (global_sound_peak <= 30) {
        strcpy(cLower, "QUIET");
    } else if (global_sound_peak <= 65) {
        strcpy(cLower, "MODERATE");
    } else if (global_sound_peak <= 85) {
        strcpy(cLower, "NOISY");
    } else {
        strcpy(cLower, "LOUD!");
    }
    break;

case 19: // J: CO2 / NH3
    dtostrf(global_co2, 4, 0, fBuf1);
    dtostrf(global_nh3, 3, 1, fBuf2);
    snprintf(cUpper, sizeof(cUpper), "CO2:%sppm", trimLeadingSpaces(fBuf1));

```

```

    snprintf(cLower, sizeof(cLower), "NH3:%sppm", trimLeadingSpaces(fBuf2));
    break;

case 20: { // K: SMOKE (avg of PM2.5+PM10) / NOX
    int smokeVal = (int)round((pmsData.pm2_5 + pmsData.pm10) / 2.0);
    dtostrf(global_nox, 3, 2, fBuf2);
    snprintf(cUpper, sizeof(cUpper), "SMOKE:%dppm", smokeVal);
    snprintf(cLower, sizeof(cLower), "NOX:%sppm", trimLeadingSpaces(fBuf2));
    break;
}

case 21: { // L: PM1.0 / UPTIME
    unsigned long upSec = millis() / 1000UL;
    unsigned int uH = upSec / 3600;
    unsigned int uM = (upSec % 3600) / 60;
    unsigned int uS = upSec % 60;
    snprintf(cUpper, sizeof(cUpper), "PM1.0:%dug", pmsData.pm1_0);
    snprintf(cLower, sizeof(cLower), "UPTIME:%02u:%02u:%02u", uH, uM, uS);
    break;
}
}

// Pre-formatted packet: "<upperLine|lowerLine>"
// Nano only needs to split on '|' and draw — no numeric parsing on its side.
nanoSerial.print("<");
nanoSerial.print(cUpper);
nanoSerial.print("|");
nanoSerial.print(cLower);
nanoSerial.println(">");

currentScreen++;
if (currentScreen >= TOTAL_SCREEN) {
    currentScreen = 0;
}
}

//=====
// Publish to HiveMQ
//=====

void publishToHiveMQ() {
    if (WiFi.status() != WL_CONNECTED || !client.connected()) return;
    if (millis() - lastMsg < MQTT_PUBLISH_INTERVAL) return;
    lastMsg = millis();

    char payload[300];
    snprintf(payload, sizeof(payload),

```

```

    "{ \"temp\":%.1f,\"hum\":%.1f,\"press\":%.1f,\"
    \"pm1\":%u,\"pm25\":%u,\"pm10\":%u,\"aqi\":%d,\"
    \"co2\":%.1f,\"nh3\":%.1f,\"voc\":%.1f,\"nox\":%.1f,\"ch4\":%.1f,\"
    \"sound\":%d} ",
    global_temp, global_hum, global_press,
    (unsigned)pmsData.pm1_0, (unsigned)pmsData.pm2_5,
    (unsigned)pmsData.pm10, global_aqi,
    global_co2, global_nh3, global_voc, global_nox, global_ch4,
    global_sound_peak);

    client.publish(topic, payload);
}

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("\n=====");
    Serial.println(" ESP32 Multi-Sensor + WiFiManager + OTA");
    Serial.println("=====");

    // Nano serial must be up before WiFiManager runs, so boot screens can display
    // during the WiFi connect/portal sequence, not just after it finishes.
    nanoSerial.begin(9600, SERIAL_8N1, 5, 4);

    sendBootScreen("WELCOME TO", "NITTTR BHOPAL", 2000);

    WiFi.mode(WIFI_STA);
    WiFiManager wm;
    wm.setAPCallback(configModeCallback); // sends "WIFI PORTAL /
    AQI_NITTTR" if config portal opens

    bool res = wm.autoConnect("AQI_NITTTR");
    if(!res) {
        delay(3000);
        ESP.restart();
    }

    WiFi.setSleep(false);

    sendBootScreen("WIFI OK!", "CONNECTED", 2000);

    sendBootScreen("SYNCING NTP", "PLEASE WAIT", 300);
    configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
    struct tm bootTimeinfo;
    bool gotTime = false;
    for (int i = 0; i < 20 && !gotTime; i++) {

```

```

    gotTime = getLocalTime(&bootTimeinfo, 500);
}
char bootTimeStr[20], bootDateStr[20];
if (gotTime) {
    getTimeDateStrings(bootTimeStr, sizeof(bootTimeStr), bootDateStr,
sizeof(bootDateStr));
} else {
    strcpy(bootDateStr, "SYNC FAILED");
}
sendBootScreen("NTP SYNCED", bootDateStr, 2000);

sendBootScreen("SYSTEM READY", "NITTTR", 2000);

ArduinoOTA.setHostname("ESP32-Env-Monitor");
ArduinoOTA.setPassword("Nitttr2026");

// The watchdog must not fire while OTA is writing flash. A flash erase/write
// cycle can block loop() for longer than WDT_TIMEOUT_SEC, which panics
the
// watchdog and resets the board mid-transfer -> "connection forcibly closed"
// on the uploader side. Pause supervision for the duration of the OTA update.
ArduinoOTA.onStart([]) {
    otaInProgress = true;

    // Flash erase/write causes a brief voltage dip that can falsely trigger
    // the brownout detector and reset the board mid-upload. Disable it only
    // for the duration of the OTA write.
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

    // Free the RAM held by the MQTT/TLS session so OTA has headroom for its
    // own write buffer — WiFiClientSecure can hold 30-40KB while connected.
    client.disconnect();
    espClient.stop();

    esp_task_wdt_delete(NULL); // stop watching this task
    Serial.println("[OTA] Start - watchdog paused, brownout disabled, MQTT
freed");
});
ArduinoOTA.onEnd([]) {
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 1); // re-enable
brownout protection
    esp_task_wdt_add(NULL); // resume normal watchdog supervision
    otaInProgress = false;
    Serial.println("[OTA] End - watchdog resumed, brownout re-enabled");
});
ArduinoOTA.onError([](ota_error_t error) {

```



```

    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 1); // re-enable
    brownout protection
    esp_task_wdt_add(NULL);    // make sure watchdog resumes even if OTA fails
    otaInProgress = false;
    Serial.printf("[OTA] Error[%u] - watchdog resumed, brownout re-enabled\n",
error);
    });

    ArduinoOTA.begin();

    pmsSerial.begin(9600, SERIAL_8N1, 16, 17);
    Wire.begin(21, 22);

    if (!bme.begin()) {
        Serial.println("[BME680] Sensor missing!");
    } else {
        bme.setTemperatureOversampling(BME680_OS_8X);
        bme.setHumidityOversampling(BME680_OS_2X);
        bme.setPressureOversampling(BME680_OS_4X);
        bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
        bme.setGasHeater(320, 150);
    }

    pinMode(MAX4466_PIN, INPUT);
    MQ135.setRegressionMethod(1);
    MQ135.init();
    mq135_cal_started = true;
    mq135_cal_timer = millis();

    espClient.setInsecure();
    client.setKeepAlive(60);
    client.setTimeout(15);
    client.setServer(mqtt_server, mqtt_port);

    // Watchdog: if loop() ever stalls for WDT_TIMEOUT_SEC seconds, ESP32
    auto-restarts
    // ESP32 core 3.x uses a config struct instead of (seconds, panic) arguments
    esp_task_wdt_config_t twdt_config = {
        .timeout_ms = WDT_TIMEOUT_SEC * 1000,
        .idle_core_mask = 0,
        .trigger_panic = true,
    };
    esp_task_wdt_init(&twdt_config);
    esp_task_wdt_add(NULL);
}

```

```

void loop() {
  ArduinoOTA.handle();

  // While a firmware write is in flight, do nothing else: no sensor polling,
  // no MQTT/serial traffic, no watchdog reset (the task was deregistered in
  // onStart()). This keeps loop() free to service ArduinoOTA.handle() as
  // fast as possible and avoids touching a watchdog handle that no longer exists.
  if (otaInProgress) {
    delay(1);
    return;
  }

  if (WiFi.status() == WL_CONNECTED) {
    if (!client.connected()) {
      reconnectMQTT();
    }
    client.loop();
  }

  handleMQ135Calibration();
  handleMQ135Read();
  handlePMS();
  handleBME680();
  handleMAX4466();

  sendScreenToNano();
  publishToHiveMQ();

  unsigned long now = millis();
  if (now - lastHeapPrint > 10000) {
    lastHeapPrint = now;
    Serial.printf("Free heap: %d\n", ESP.getFreeHeap());
  }

  // Feed the watchdog every loop — proves the board is alive and not hung
  esp_task_wdt_reset();

  // Scheduled restart every 24h to clear any long-term heap fragmentation.
  // WiFiManager reconnects automatically with saved credentials; MQTT
  reconnects via reconnectMQTT().
  if (now >= RESTART_INTERVAL_MS) {
    Serial.println("[MAINTENANCE] 24h uptime reached, restarting for
    stability...");
    delay(500);
    ESP.restart();
  }
}

```

```

    delay(1);
}

```

THE FOLLOWING CODE IS THE CODE FOR ARDUINO NANO:

```

/*
 * Arduino Nano - 10 Chained P10 Panels (Serpentine Wiring)
 * Layout: 5->4->3->2->1 (top row) then 6->7->8->9->10 (bottom row)
 * Role: PURE DISPLAY DRIVER ONLY.
 * All formatting/switching logic lives on the ESP32 now.
 * Nano just receives "<upperLine|lowerLine>" and draws it. Nothing else.
 */

#include <SPI.h>
#include <DMD.h>
#include <TimerOne.h>
#include "Arial_Black_16.h"
#include <avr/wdt.h>

// ——— P10 PANEL CONFIGURATION
—————

#define DISPLAYS_WIDE 5 // 5 panels wide (160 pixels)
#define DISPLAYS_HIGH 2 // 2 panels stacked (32 pixels)
DMD dmd(DISPLAYS_WIDE, DISPLAYS_HIGH);

// ——— SERIAL RECEIVE BUFFER (small — only two short text lines now)
—————

const byte numChars = 64;
char receivedChars[numChars];
boolean newData = false;

// ——— CURRENT SCREEN TEXT (persists until next valid packet arrives)
—————

char cUpper[32] = "CONNECTING";
char cLower[32] = "PLEASE WAIT";

void ScanDMD() {
    dmd.scanDisplayBySPI();
}

void setup() {

```

```

    // MUST be first two lines: clears watchdog reset flag and disables WDT
    immediately.
    // Without this, a watchdog-triggered reset can re-trigger before setup() finishes,
    // causing the Nano to loop-reset forever and never actually draw anything.
    MCUSR = 0;
    wdt_disable();

    Serial.begin(9600); // Same UART line from ESP32 (Serial1 / GPIO4-5 on ESP32
    side)

    Timer1.initialize(5000);
    Timer1.attachInterrupt(ScanDMD);

    dmd.clearScreen(true);
    dmd.selectFont(Arial_Black_16);

    // Show initial placeholder immediately so panel isn't blank before first packet
    drawCurrentScreen();

    // Watchdog: if loop() ever hangs for 8s (e.g. stuck serial read), Nano auto-resets.
    // Safe here since Timer1's ISR (ScanDMD) keeps the panel refreshed
    independently of loop().
    wdt_enable(WDTO_8S);
}

void loop() {
    wdt_reset(); // Feed watchdog every pass — proves loop() is alive, not frozen

    recvWithStartEndMarkers();

    if (newData) {
        processPacket();
        newData = false;
    }
}

// ——— Non-blocking Packet Reader (same bulletproof recovery as before)
—————

void recvWithStartEndMarkers() {
    static boolean recvInProgress = false;
    static byte ndx = 0;
    char startMarker = '<';
    char endMarker = '>';
    char rc;

    while (Serial.available() > 0 && newData == false) {

```

```

rc = Serial.read();

if (recvInProgress) {
  if (rc != endMarker) {
    receivedChars[ndx] = rc;
    ndx++;
    if (ndx >= numChars) {
      ndx = numChars - 1; // Clamp instead of wrapping to avoid overwrite
      corruption
    }
  } else {
    receivedChars[ndx] = '\0';
    recvInProgress = false;
    ndx = 0;
    newData = true;
  }
} else if (rc == startMarker) {
  recvInProgress = true;
  ndx = 0;
}
}

// —— Split "upperLine|lowerLine" and update display text


---


void processPacket() {
  char *sep = strchr(receivedChars, '|');
  if (sep == NULL) {
    return; // Malformed packet — keep showing the last good screen, don't corrupt
    it
  }
  *sep = '\0';

  strncpy(cUpper, receivedChars, sizeof(cUpper) - 1);
  cUpper[sizeof(cUpper) - 1] = '\0';

  strncpy(cLower, sep + 1, sizeof(cLower) - 1);
  cLower[sizeof(cLower) - 1] = '\0';

  drawCurrentScreen();
}

// —— Draw cUpper/cLower to the panel (instant pop-up, no animation)


---


void drawCurrentScreen() {
  dmd.clearScreen(true);

```

```

    dmd.drawString(4, 16, cUpper, strlen(cUpper), GRAPHICS_NORMAL); //
bottom row
    dmd.drawString(4, 0, cLower, strlen(cLower), GRAPHICS_NORMAL); // top
row
}

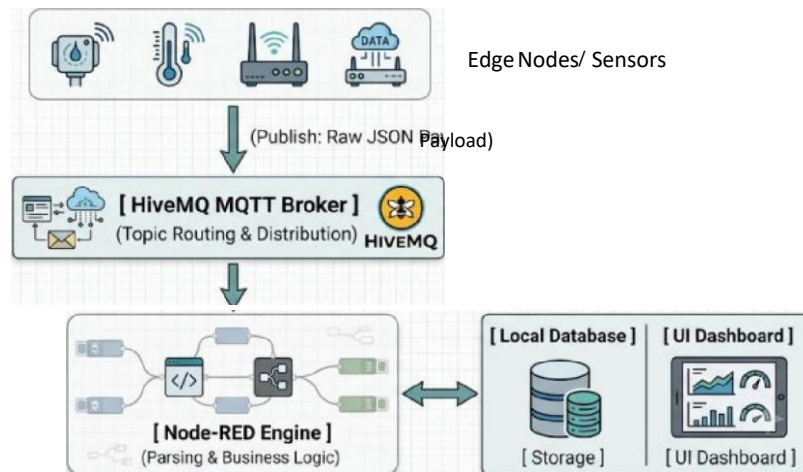
```

4 .DATA FLOW & LOCAL SERVER

4.1 DATA FLOW:

Data flow defines the structured pathway that information travels throughout the system—from edge devices and sensors to the processing core, and finally to the user interface or storage layer. In this architecture, data moves asynchronously and is heavily event-driven to ensure real-time responsiveness and efficient bandwidth utilization.

To maintain system integrity, data validation and error-checking mechanisms are implemented at various stages of the transmission lifecycle. The standard lifecycle of data flowing through this system consists of the following four sequential phases:



- **Data Generation & Publication:** Edge nodes, microcontrollers, or software clients capture environmental, telemetry, or system variables. This raw data is packaged into lightweight payloads (typically formatted in standard JSON for interoperability) and published to specific, predetermined hierarchical topics.
- **Broker Interception & Routing:** The published data is received by the centralized broker. The broker evaluates the topic metadata in real time, cross-references it with active client subscriptions, and immediately routes the payload to all components that have subscribed to that specific topic.
- **Ingestion, Parsing & Orchestration:** The backend logic engine intercepts the incoming data stream from the broker. Here, the data is visually routed through predefined logical flows. The raw payload is parsed, cleaned of any anomalies, and transformed into a standardized format suitable for application processing.
- **Consumable Output & Persistence:** The processed data is pushed to its final destination. Depending on the system state, it is either committed to a local database for historical logging, forwarded

to a local dashboard for live user visualization, or used to trigger automated hardware/actuator actions back at the edge.

4.2 LOCAL SERVER

The local server serves as the localized physical or virtual infrastructure that hosts the core backend services of the project. By running these services on a local machine (such as a dedicated PC, workstation, or single-board computer like a Raspberry Pi) using a loopback interface (localhost or 127.0.0.1), the system eliminates reliance on external cloud environments.

This localized deployment strategy offers several critical architectural advantages: •
Network Independence: The system remains fully operational offline or within an isolated Local Area Network (LAN), mitigating risks associated with internet downtime.

- Zero Cloud Costs: Eliminates monthly subscription fees for cloud hosting, data ingestion, and compute cycles.
- Enhanced Security: Sensitive project data remains localized to the host machine, drastically reducing the attack surface by avoiding exposure to the public internet.

4.2.1 HIVEMQ

HiveMQ is utilized within the local architecture as the enterprise-grade MQTT (Message Queuing Telemetry Transport) broker. It acts as the central communication hub, managing the connections of all clients and handling the distribution of messages.

- Role in the System: HiveMQ facilitates completely decoupled, bidirectional communication. Instead of clients communicating directly with each other (which creates complex, tightly coupled dependencies), they communicate solely through HiveMQ using a publish/subscribe model. This ensures that if one node goes offline, the rest of the system continues to function normally.
- Key Technical Features Utilized:
 - High Throughput & Low Latency: Designed for high-performance messaging, HiveMQ handles fast message delivery with minimal CPU and memory overhead on the local server hardware.
 - Topic Trees: It uses hierarchical topic structures (e.g., factory/machine/temperature) to organize data channels dynamically, allowing for scalable data routing.
- Quality of Service (QoS): It ensures message delivery reliability across the local network by supporting various QoS levels (QoS 0: At most once, QoS 1: At least once, QoS 2: Exactly once) depending on how critical the data point is.

4.2.2 NODE-RED

Node-RED is a flow-based, low-code development tool used as the primary middleware logic engine on the local server. Built on the Node.js runtime, it operates via a browser-based flow editor and excels at wiring together hardware devices, APIs, and online services.

- Role in the System: Node-RED connects the communication broker (HiveMQ) to the rest of the application ecosystem. It acts as the event-driven "brain" that intercepts MQTT messages, applies business logic, and routes them to dashboards, alerts, or databases.

- **Key Functions Performed:**
- **Data Transformation & Filtering:** It parses raw payloads using JavaScript function nodes to extract key variables, filter out noise, and convert units (e.g., Celsius to Fahrenheit) if necessary.
- **System Integration:** It bridges different protocols, APIs, and file systems using a vast ecosystem of pre-built nodes, allowing the project to scale without writing extensive boilerplate code.
- **UI Dashboarding:** Node-RED hosts localized UI dashboards (node-reddashboard), rendering real-time graphs, gauges, and control switches to the user directly from the local server's IP address.

